Power BI Visualization Guide

This document explains how to recreate the CampaignWe HTML dashboard in Power BI Desktop using the parquet files produced by `process_campaignwe.py`.

> **Data note**: The parquet files contain click data that has been pre-enriched with organisational (HR) fields during processing. The data is anonymised — no personally identifiable information is included. You do not need to perform any data matching yourself.

Table of Contents

1. Data Import
2. Data Model & Relationships
3. Calculated Columns (Power Query)
4. DAX Measures
5. Page 1 — Overview
6. Page 2 — Divisions & Regions
7. Page 3 — Stories
8. Page 4 — Data Completeness
9. Slicers & Cross-Filtering
10. Appendix — Full DAX Reference

---

1. Data Import

Parquet Files

Power BI Desktop can import parquet files natively (since the February 2023 release). The files live in the agreed SharePoint folder.

File	Grain	Description
`events_anonymized.parquet`	One row per click event	Anonymised click data with organisational fields — primary source for all visuals
`story_metadata.parquet`	One row per story	Story lookup table with story text, author info, and keys

Import Steps

1. **Get Data → Parquet**
    - Home → Get Data → More → Parquet
    - Browse to `events_anonymized.parquet` → Load
    - Repeat for `story_metadata.parquet` → Load
2. **Rename tables** in the Model view:

    - `events_anonymized` → **Events**
    - `story_metadata` → **StoryMeta**

3. **Check column types** in Power Query Editor (Transform Data):
    - Events: `session_date` / `date` → **Date**
    - Events: `timestamp`, `timestamp_cet` → **DateTime**
    - Events: `event_hour`, `event_weekday_num`, `story_id` → **Whole Number**
    - Events: `person_hash` → **Text**
    - Events: All `hr_*` columns → **Text**
    - Events: All count columns → **Whole Number**
    - StoryMeta: `story_id` → **Text** (to match Events[story_id] after cast)
    - StoryMeta: `author_email`, `author_division`, `author_department`, `author_job_title` → **Text**

All dashboard visuals run against event-level data. The StoryMeta table provides story labels and author information.

---

2. Data Model & Relationships

Date Table (Required for Time Intelligence)

Power BI needs a proper date table for time-intelligence DAX functions. Create one:

```
DateTable =
ADDCOLUMNS(
    CALENDARAUTO(),
    "Year", YEAR([Date]),
    "Month", FORMAT([Date], "MMMM"),
    "MonthNum", MONTH([Date]),
    "WeekdayName", FORMAT([Date], "dddd"),
    "WeekdayNum", WEEKDAY([Date], 2),
    "YearMonth", FORMAT([Date], "YYYY-MM")
)
```

Mark it as the date table: Model view → right-click DateTable → "Mark as date table" → select the `Date` column.

Hour Table (For Heatmap)

```
HourTable = GENERATESERIES(0, 23, 1)
```

Rename the column to `Hour`.

Relationships

From	To	Cardinality	Key
Events[session_date]	DateTable[Date]	Many-to-One	Active
Events[event_hour]	HourTable[Hour]	Many-to-One	Active
Events[story_id]	StoryMeta[story_id]	Many-to-One	Active

Set cross-filter direction to **Single** for all relationships.

---

3. Calculated Columns (Power Query)

These columns already exist in the parquet file from the Python pipeline, so you should **not** need to recreate them. Verify they are present:

Column	Description	Already in Parquet?
`session_date`	CET-based date	Yes
`event_hour`	Hour in CET (0–23)	Yes
`event_weekday`	Day name (Monday–Sunday)	Yes
`event_weekday_num`	ISO weekday (1=Mon, 7=Sun)	Yes
`action_type`	Read, Like, Open Form, Submit, Cancel, Other	Yes
`story_id`	Extracted story number	Yes
`session_key`	Unique session identifier	Yes
`person_hash`	Anonymised user identifier (hash)	Yes
`hr_division` through `hr_function`	Organisational hierarchy fields	Yes
`hr_region`, `hr_country`	Geographic fields	Yes

If any column is missing, add it in Power Query (Transform Data) using M formulas equivalent to the Python logic documented in data-pipeline.md.

---

4. DAX Measures

Create a dedicated **Measures** table (Enter Data → empty table → rename to `_Measures`). Place all measures here for cleanliness.

Core KPIs

```
Total Clicks = COUNTROWS(Events)
```

```
Unique Visitors = DISTINCTCOUNT(Events[person_hash])

Unique Sessions = DISTINCTCOUNT(Events[session_key])

Unique Stories = DISTINCTCOUNT(Events[story_id])

Clicks per Visitor =
DIVIDE(
    [Total Clicks],
    [Unique Visitors],
    0
)

Org Coverage % =
DIVIDE(
    COUNTROWS(FILTER(Events, NOT(ISBLANK(Events[hr_division])))),
    [Total Clicks],
    0
) * 100
```

Action Type Counts

```
Views = CALCULATE([Total Clicks], Events[action_type] = "Read")

Likes = CALCULATE([Total Clicks], Events[action_type] = "Like")

Open Forms = CALCULATE([Total Clicks], Events[action_type] = "Open Form")

Submits = CALCULATE([Total Clicks], Events[action_type] = "Submit")

Cancels = CALCULATE([Total Clicks], Events[action_type] = "Cancel")
```

Engagement Metrics

```
Views per Visitor =
DIVIDE(
    [Views],
    [Unique Visitors],
    0
)

Clicks per Visitor (Division Table) =
DIVIDE(
    [Total Clicks],
    [Unique Visitors],
    0
)
```

Data Completeness Metrics

```
Org Matched Count =
COUNTROWS(
    FILTER(Events,
        NOT(ISBLANK(Events[person_hash])) &&
NOT(ISBLANK(Events[hr_division]))
    )
)

User No Org Count =
COUNTROWS(
    FILTER(Events,
        NOT(ISBLANK(Events[person_hash])) && ISBLANK(Events[hr_division])
    )
)

No User Count =
COUNTROWS(
    FILTER(Events, ISBLANK(Events[person_hash]))
)
```

Percentage Measures (for bar/doughnut labels)

```
% of Total Clicks =
DIVIDE(
    [Total Clicks],
    CALCULATE([Total Clicks], REMOVEFILTERS()),
    0
)
```

---

5. Page 1 — Overview

Layout

```
┌──────────────────────────────────────────────────────────────────────┐
│  [Date Slicer]  [Action Type Slicer]  [Link Type Slicer]             │
├──────────────┬──────────────┬──────────────┬──────────────┬──────────┤
│  Total       │ Unique       │  Unique      │  Unique      │ Clicks / │   Org    │
│  Clicks      │ Visitors     │ Sessions     │ Stories      │ Visitor  │ Coverage │
├──────────────┴──────────────┴──────────────┴──────────────┴──────────┤
│              Daily Activity Trend (line + area)                      │
├──────────────────────────────────────┬──────────────────────────────┤
│     Clicks by Hour (bar)             │  Clicks by Weekday (bar)     │
├──────────────────────────────────────┼──────────────────────────────┤
│  Activity Heatmap (Weekday×Hour)     │  Action Types (doughnut)     │
```

```
├──────────────────────────────────────┼──────────────────────────────────────┤
│                Link Types (horizontal bar)                                   │
└──────────────────────────────────────────────────────────────────────────────┘
```

6.1 KPI Cards

Add six **Card** visuals (or a single **Multi-row Card**) across the top:

Card	Measure	Format
Total Clicks	`[Total Clicks]`	Whole number, thousands separator
Unique Visitors	`[Unique Visitors]`	Whole number
Unique Sessions	`[Unique Sessions]`	Whole number
Unique Stories	`[Unique Stories]`	Whole number
Clicks/Visitor	`[Clicks per Visitor]`	1 decimal
Org Coverage	`[Org Coverage %]`	1 decimal + "%" suffix

Formatting: Background = `#ECEBE4`, font color = `#000000`, callout value color = `#E60000`.

6.2 Daily Activity Trend

Visual type: Line chart (combo chart: area + line)

Setting	Value
X-axis	DateTable[Date]
Y-axis (Column series)	`[Total Clicks]` — set as **Area**
Y-axis (Line series)	`[Unique Visitors]` — set as **Line**, dashed
Area color	`#404040` with ~30% opacity
Line color	`#B8B3A2` (Unique Visitors)
Line style	Dashed
Secondary Y-axis	On (for Unique Visitors)

6.3 Clicks by Hour

Visual type: Clustered bar chart (vertical)

Setting	Value
X-axis	Events[event_hour] (or HourTable[Hour])
Y-axis	`[Total Clicks]`
Bar color	`#5A5D5C`

Setting	Value
Data labels	On — show value

To also show percentage, add a tooltip measure:

```
Hour % of Total =
DIVIDE(
    [Total Clicks],
    CALCULATE([Total Clicks], REMOVEFILTERS(Events[event_hour])),
    0
)
```

Enable **Data labels** and add `[Hour % of Total]` as tooltip.

6.4 Clicks by Weekday

Visual type: Clustered bar chart (vertical)

Setting	Value
X-axis	Events[event_weekday] — sort by Events[event_weekday_num]
Y-axis	`[Total Clicks]`
Data labels	On

Color by weekday/weekend: Use conditional formatting with rules:

```
Weekday Color =
IF(
    MAX(Events[event_weekday_num]) >= 6,
    "#CCCABC",
    "#5A5D5C"
)
```

Apply via Format → Data colors → fx (conditional formatting) → Field value → `[Weekday Color]`.

> **Sort order**: Click on the weekday axis, then in the column tools ribbon, "Sort by Column" → select `event_weekday_num` to ensure Monday–Sunday order.

6.5 Activity Heatmap (Weekday x Hour)

Visual type: Matrix

Setting	Value
Rows	Events[event_weekday] (sorted by event_weekday_num)

Setting	Value
Columns	Events[event_hour] (or HourTable[Hour])
Values	`[Total Clicks]`

Conditional formatting (on the values):

- Format → Cell elements → Background color → fx
- Format style: Gradient
- Minimum: `#FFFFFF`
- Center: `#E4A911`
- Maximum: `#8A000A`
- Based on: `[Total Clicks]`

Disable row and column totals for a cleaner heatmap look. Set font size small (8pt) to fit 24 columns.

6.6 Action Types Doughnut

Visual type: Donut chart

Setting	Value
Legend	Events[action_type]
Values	`[Total Clicks]`
Detail labels	Category, Value, Percent of total

Colors will auto-assign from the theme's `dataColors` array. To handle blanks, create a calculated column:

```
// In Power Query or as a calculated column:
Action Type Display = IF(ISBLANK(Events[action_type]), "(null)",
Events[action_type])
```

Use `Action Type Display` as the legend field.

6.7 Link Types (Horizontal Bar)

Visual type: Clustered bar chart (horizontal)

Setting	Value
Y-axis	Events[CP_Link_Type]
X-axis	`[Total Clicks]`
Bar color	`#5A5D5C`
Data labels	On
Sort	Descending by value

Handle blanks similarly:

```
Link Type Display = IF(ISBLANK(Events[CP_Link_Type]), "(blank)",
Events[CP_Link_Type])
```

---

6. Page 2 — Divisions & Regions

Layout

```
┌──────────────────────────────────────────────────────────────┐
│  [Date Slicer]  [Action Type Slicer]  [Link Type Slicer]      │
├──────────────────────────────┬───────────────────────────────┤
│  Division Drilldown (bar)    │  Region → Country (bar)       │
├──────────────────────────────┴───────────────────────────────┤
│        Daily Visitors — Top 5 Divisions (multi-line)          │
├──────────────────────────────────────────────────────────────┤
│                   Division Summary Table                      │
└──────────────────────────────────────────────────────────────┘
```

6.1 Division Drilldown (GCRS Hierarchy)

Visual type: Clustered bar chart with **drill-down hierarchy**

Create a hierarchy on the Events table:

1. Right-click `hr_division` → New hierarchy → rename to "GCRS Hierarchy"
2. Drag into the hierarchy in order:
   - `hr_division`
   - `hr_unit`
   - `hr_area`
   - `hr_sector`
   - `hr_segment`
   - `hr_function`

Setting	Value
X-axis	GCRS Hierarchy
Y-axis	`[Total Clicks]` and `[Unique Visitors]` (grouped)
Bar colors	`#404040` (Clicks), `#B8B3A2` (Unique Visitors)
Drill mode	Enable drill-down (↓ icon in visual header)
Top N	Optional: filter to Top 20 by `[Total Clicks]`

Drill behavior: Users click the drill-down arrow, then click a bar to drill into the next GCRS level. The breadcrumb appears automatically at the top of the visual.

6.2 Region → Country Drilldown

Visual type: Clustered bar chart with drill-down

Create a hierarchy:

1. Right-click `hr_region` → New hierarchy → "Geography"
2. Add `hr_country` beneath it

Setting	Value
X-axis	Geography hierarchy
Y-axis	`[Total Clicks]` and `[Unique Visitors]` (grouped)
Bar colors	`#404040` (Clicks), `#B8B3A2` (Unique Visitors)
Drill mode	Enable drill-down

6.3 Daily Visitors — Top 5 Divisions

Visual type: Line chart

This requires a Top N filter to show only the 5 most active divisions.

Setting	Value
X-axis	DateTable[Date]
Y-axis	`[Unique Visitors]`
Legend	Events[hr_division]

Top N filter: Click on the visual → Filters pane → `hr_division` → Filter type: Top N → Top 5 by `[Unique Visitors]`.

Colors will auto-assign from the 20-color theme palette.

6.4 Division Summary Table

Visual type: Table

Column	Value/Measure
Division	Events[hr_division]
Clicks	`[Total Clicks]`
Unique Visitors	`[Unique Visitors]`
Clicks/Visitor	`[Clicks per Visitor]`
Stories	`[Unique Stories]`
Views	`[Views]`

Column	Value/Measure
Likes	`[Likes]`

Formatting:

- Style preset: Alternating rows
- Row background alt: `#F8F7F2`
- Header background: `#ECEBE4`
- Sort default: Clicks descending

---

7. Page 3 — Stories

Layout

```
┌──────────────────────────────────────────────────────────────┐
│  [Date Slicer]  [Action Type Slicer]  [Link Type Slicer]      │
├──────────────────────────────────┬──────────────────────────────┤
│  Top Stories by Views (bar)      │  Top Stories by Visitors (bar) │
├──────────────────────────────────┴──────────────────────────────┤
│          Engagement Funnel — Top 10 Stories (grouped bar)       │
├──────────────────────────────────┬──────────────────────────────┤
│  Division × Story Heatmap        │  Region × Story Heatmap      │
├──────────────────────────────────┴──────────────────────────────┤
│         Daily Views — Top 5 Stories (multi-line)                │
└──────────────────────────────────────────────────────────────┘
```

7.1 Top Stories by Views

Visual type: Clustered bar chart (horizontal)

Setting	Value
Y-axis	Events[story_id]
X-axis	`[Views]`
Bar color	`#5A5D5C`
Top N filter	Top 20 by `[Views]`
Data labels	On
Sort	Descending by value

Label formatting: Create a calculated column that shows the story title when available (the dev team will add this field to the SharePoint list), falls back to the author's email, and finally to "Story {id}":

```
Story Label =
IF(
```

```
    ISBLANK(Events[story_id]),
    BLANK(),
    VAR _title = RELATED(StoryMeta[story_title])
    VAR _author = RELATED(StoryMeta[author_email])
    RETURN COALESCE(_title, _author, "Story " & Events[story_id])
)
```

Use `Story Label` on the Y-axis. Filter out blanks (where story_id is null).

7.2 Top Stories by Unique Visitors

Visual type: Clustered bar chart (horizontal)

```
Unique Visitors =
CALCULATE(
    DISTINCTCOUNT(Events[person_hash]),
    Events[action_type] = "Read"
)
```

Setting	Value
Y-axis	Story Label
X-axis	`[Unique Visitors]`
Bar color	`#5A5D5C`
Top N filter	Top 20 by `[Unique Visitors]`
Data labels	On

7.3 Engagement Funnel — Top 10 Stories

Visual type: Clustered bar chart (vertical, grouped)

Setting	Value
X-axis	Story Label
Y-axis	`[Views]`, `[Likes]`
Top N filter	Top 10 by `[Views]` on story_id
Legend	Measure names

Assign distinct colors from the chart palette to each measure series:

- Views → `#404040`
- Likes → `#6F7A1A`

> **Alternative approach**: If having multiple measures on one axis is cumbersome, unpivot the action types into a single column using a DAX summary table. See the Appendix for the `StoryFunnel`

> table pattern.

7.4 Division x Story Heatmap

Visual type: Matrix

Setting	Value
Rows	Events[hr_division]
Columns	Story Label
Values	`[Views]`

Top N filters:

- Rows: Top 10 hr_division by `[Total Clicks]`
- Columns: Top 10 story_id by `[Views]`

Conditional formatting: Same heatmap gradient as the Activity Heatmap (`#FFFFFF` → `#E4A911` → `#8A000A`).

7.5 Region x Story Heatmap

Visual type: Matrix — identical to Division heatmap but with `hr_region` on rows.

7.6 Daily Views — Top 5 Stories

Visual type: Line chart

```
Daily Views =
CALCULATE(
    COUNTROWS(Events),
    Events[action_type] = "Read"
)
```

Setting	Value
X-axis	DateTable[Date]
Y-axis	`[Daily Views]`
Legend	Story Label
Top N filter	Top 5 story_id by `[Views]`

---

8. Page 4 — Data Completeness

Layout

```
┌──────────────────────────────────────────────────────────────┐
│  [Date Slicer]                                               │
├──────────────────────────────────────────────────────────────┤
│          Organisational Data Coverage (stacked bar)           │
├──────────────────────────────────────────────────────────────┤
│                    Field Coverage Table                      │
└──────────────────────────────────────────────────────────────┘
```

8.1 Organisational Data Coverage

Visual type: Clustered bar chart (horizontal, 3 bars)

Shows what proportion of clicks have organisational fields populated (division, region, etc.).

Create a helper table:

```
CoverageCategory =
DATATABLE(
    "Category", STRING, "SortOrder", INTEGER,
    {
        {"Org Data Available", 1},
        {"User Known, No Org Data", 2},
        {"Unknown User", 3}
    }
)
```

And a measure that switches by category:

```
Coverage Count =
SWITCH(
    SELECTEDVALUE(CoverageCategory[Category]),
    "Org Data Available", [Org Matched Count],
    "User Known, No Org Data", [User No Org Count],
    "Unknown User", [No User Count],
    0
)
```

Setting	Value
Y-axis	CoverageCategory[Category] (sorted by SortOrder)
X-axis	`[Coverage Count]`
Data labels	On — value and percentage

Manual colors:

- Org Data Available → `#6F7A1A` (RAG Green)

- User Known, No Org Data → `#E4A911` (RAG Amber)
- Unknown User → `#CCCABC` (Light gray)

8.2 Field Coverage Table

Visual type: Table (or Matrix)

Shows null rates per field. Use a disconnected field list table and SWITCH:

```
FieldList =
DATATABLE(
    "FieldName", STRING,
    {
        {"person_hash"}, {"session_id"}, {"user_id"},
        {"story_id"}, {"action_type"},
        {"hr_division"}, {"hr_unit"}, {"hr_area"},
        {"hr_region"}, {"hr_country"}
    }
)
```

```
Field Non-Null Count =
VAR _field = SELECTEDVALUE(FieldList[FieldName])
RETURN SWITCH(
    _field,
    "person_hash", COUNTROWS(FILTER(Events,
NOT(ISBLANK(Events[person_hash])))),
    "session_id", COUNTROWS(FILTER(Events,
NOT(ISBLANK(Events[session_id])))),
    "user_id", COUNTROWS(FILTER(Events, NOT(ISBLANK(Events[user_id])))),
    "story_id", COUNTROWS(FILTER(Events, NOT(ISBLANK(Events[story_id])))),
    "action_type", COUNTROWS(FILTER(Events,
NOT(ISBLANK(Events[action_type])))),
    "hr_division", COUNTROWS(FILTER(Events,
NOT(ISBLANK(Events[hr_division])))),
    "hr_unit", COUNTROWS(FILTER(Events, NOT(ISBLANK(Events[hr_unit])))),
    "hr_area", COUNTROWS(FILTER(Events, NOT(ISBLANK(Events[hr_area])))),
    "hr_region", COUNTROWS(FILTER(Events,
NOT(ISBLANK(Events[hr_region])))),
    "hr_country", COUNTROWS(FILTER(Events,
NOT(ISBLANK(Events[hr_country])))),
    BLANK()
)

Field Null Count = [Total Clicks] - [Field Non-Null Count]

Field Coverage % = DIVIDE([Field Non-Null Count], [Total Clicks], 0) * 100
```

Column	Value

Column	Value
Field	FieldList[FieldName]
Non-Null	`[Field Non-Null Count]`
Null	`[Field Null Count]`
Coverage %	`[Field Coverage %]`

---

9. Slicers & Cross-Filtering

Date Range Slicer

Add a **Date slicer** (Between mode) connected to `DateTable[Date]`.

Power BI does not natively support named presets (7d, 14d, 30d, etc.), but you can add relative date filtering:

- Click the slicer → Filter type → Relative date → "Last 7/14/30 days"
- Or use a **Bookmark** for each preset (View → Bookmarks → add one per range)

Action Type Slicer

Visual type: Slicer (list or dropdown)

Setting	Value
Field	Events[action_type] (or Action Type Display)
Selection	Single select or multi-select
Style	Tile or List

Link Type Slicer

Visual type: Slicer

Setting	Value
Field	Events[CP_Link_Type] (or Link Type Display)
Style	Dropdown (saves space)

Cross-Filtering Behavior

By default, Power BI cross-filters between visuals on the same page. This mirrors the HTML dashboard's click-to-filter behavior:

- Clicking a doughnut slice filters the entire page to that action type
- Clicking a bar in Link Types filters to that link type
- Clicking a division bar filters stories, etc.

To control cross-filtering: select a visual → Format → Edit interactions → choose Filter/Highlight/None for each other visual.

Recommended interactions:

- Action Type doughnut → **Filter** all other visuals
- Link Type bar → **Filter** all other visuals
- Division bar → **Filter** story visuals (when on same page)
- Date slicer → **Filter** everything

---

10. Appendix — Full DAX Reference

All Measures in One Block

```
// ═══════════════════════════════════════════
// CORE KPIs
// ═══════════════════════════════════════════

Total Clicks = COUNTROWS(Events)

Unique Visitors = DISTINCTCOUNT(Events[person_hash])

Unique Sessions = DISTINCTCOUNT(Events[session_key])

Unique Stories = DISTINCTCOUNT(Events[story_id])

Clicks per Visitor =
DIVIDE([Total Clicks], [Unique Visitors], 0)

Org Coverage % =
DIVIDE(
    COUNTROWS(FILTER(Events, NOT(ISBLANK(Events[hr_division])))),
    [Total Clicks],
    0
) * 100


// ═══════════════════════════════════════════
// ACTION TYPE COUNTS
// ═══════════════════════════════════════════

Views = CALCULATE([Total Clicks], Events[action_type] = "Read")

Likes = CALCULATE([Total Clicks], Events[action_type] = "Like")

Open Forms = CALCULATE([Total Clicks], Events[action_type] = "Open Form")

Submits = CALCULATE([Total Clicks], Events[action_type] = "Submit")

Cancels = CALCULATE([Total Clicks], Events[action_type] = "Cancel")
```

```
// ═══════════════════════════════════════════════
// ENGAGEMENT METRICS
// ═══════════════════════════════════════════════

Views per Visitor = DIVIDE([Views], [Unique Visitors], 0)

Unique Visitors =
CALCULATE(DISTINCTCOUNT(Events[person_hash]), Events[action_type] =
"Read")

Daily Views =
CALCULATE(COUNTROWS(Events), Events[action_type] = "Read")


// ═══════════════════════════════════════════════
// PERCENTAGE & LABEL HELPERS
// ═══════════════════════════════════════════════

% of Total Clicks =
DIVIDE([Total Clicks], CALCULATE([Total Clicks], REMOVEFILTERS()), 0)

Hour % of Total =
DIVIDE(
    [Total Clicks],
    CALCULATE([Total Clicks], REMOVEFILTERS(Events[event_hour])),
    0
)


// ═══════════════════════════════════════════════
// DATA COMPLETENESS
// ═══════════════════════════════════════════════

Org Matched Count =
COUNTROWS(
    FILTER(Events,
        NOT(ISBLANK(Events[person_hash])) &&
NOT(ISBLANK(Events[hr_division]))
    )
)

User No Org Count =
COUNTROWS(
    FILTER(Events,
        NOT(ISBLANK(Events[person_hash])) && ISBLANK(Events[hr_division])
    )
)

No User Count =
COUNTROWS(FILTER(Events, ISBLANK(Events[person_hash])))

Org Matched % = DIVIDE([Org Matched Count], [Total Clicks], 0) * 100
User No Org % = DIVIDE([User No Org Count], [Total Clicks], 0) * 100
```

```
No User % = DIVIDE([No User Count], [Total Clicks], 0) * 100


// ═══════════════════════════════════════════
// TABLE-SPECIFIC
// ═══════════════════════════════════════════

First Seen = MIN(Events[session_date])

Last Seen = MAX(Events[session_date])


// ═══════════════════════════════════════════
// CONDITIONAL FORMATTING HELPERS
// ═══════════════════════════════════════════

Weekday Color =
IF(MAX(Events[event_weekday_num]) >= 6, "#CCCABC", "#5A5D5C")
```

Calculated Columns

```
// On Events table:
Action Type Display =
IF(ISBLANK(Events[action_type]), "(null)", Events[action_type])

Link Type Display =
IF(ISBLANK(Events[CP_Link_Type]), "(blank)", Events[CP_Link_Type])

// Story Label: use story_title if available (coming soon), then
author_email, then fallback to "Story {id}"
Story Label =
IF(
    ISBLANK(Events[story_id]),
    BLANK(),
    VAR _title = RELATED(StoryMeta[story_title])
    VAR _author = RELATED(StoryMeta[author_email])
    RETURN COALESCE(_title, _author, "Story " & Events[story_id])
)

Division Display =
IF(ISBLANK(Events[hr_division]), "(unknown)", Events[hr_division])

Region Display =
IF(ISBLANK(Events[hr_region]), "(unknown)", Events[hr_region])
```

Helper Tables

```
// Date table
DateTable =
```

```
ADDCOLUMNS(
    CALENDARAUTO(),
    "Year", YEAR([Date]),
    "Month", FORMAT([Date], "MMMM"),
    "MonthNum", MONTH([Date]),
    "WeekdayName", FORMAT([Date], "dddd"),
    "WeekdayNum", WEEKDAY([Date], 2),
    "YearMonth", FORMAT([Date], "YYYY-MM")
)

// Hour table (for matrix heatmap axis)
HourTable = GENERATESERIES(0, 23, 1)

// Coverage category table
CoverageCategory =
DATATABLE(
    "Category", STRING, "SortOrder", INTEGER,
    {
        {"Org Data Available", 1},
        {"User Known, No Org Data", 2},
        {"Unknown User", 3}
    }
)

// Field list table (for field coverage)
FieldList =
DATATABLE(
    "FieldName", STRING,
    {
        {"person_hash"}, {"session_id"}, {"user_id"},
        {"story_id"}, {"action_type"},
        {"hr_division"}, {"hr_unit"}, {"hr_area"},
        {"hr_region"}, {"hr_country"}
    }
)
```

Story Funnel (Alternative for Section 7.3)

If placing multiple measures on a single bar chart is awkward, create a summary table:

```
StoryFunnel =
SUMMARIZECOLUMNS(
    Events[story_id],
    Events[action_type],
    "Count", COUNTROWS(Events)
)
```

Then use `story_id` on the X-axis, `action_type` as Legend, and `Count` as Value in a stacked/grouped bar chart. Apply a Top N visual filter on `story_id` by `[Views]`.

Quick-Start Checklist

1. ☐ Import `events_anonymized.parquet` → rename table to **Events**
2. ☐ Import `story_metadata.parquet` → rename table to **StoryMeta**
3. ☐ Create **DateTable**, **HourTable**, **CoverageCategory**, **FieldList** DAX tables
4. ☐ Set up relationships (Events → DateTable, Events → HourTable, Events → StoryMeta)
5. ☐ Create `_Measures` table and paste all DAX measures
6. ☐ Add calculated columns (Action Type Display, Story Label, etc.)
7. ☐ Build Page 1 (Overview) — KPIs, trend, hour/weekday bars, heatmap, doughnut
8. ☐ Build Page 2 (Divisions & Regions) — GCRS hierarchy, region drilldown, table
9. ☐ Build Page 3 (Stories) — top stories, funnel, heatmaps, daily trend
10. ☐ Build Page 4 (Data Completeness) — org coverage bar, field coverage table
11. ☐ Add slicers (Date, Action Type, Link Type) to each page
12. ☐ Configure cross-filter interactions between visuals
13. ☐ Test drill-down on Division and Region charts